Marvin User Manual

Complete guide to the local chat and coding workstation

Application version	1.20.0
User manual	2026

Local AI · Qwen 3.8 · Ornith 1.5 · Windows 11 · RTX 5090

Contents

1. What Marvin Is 3

2. Requirements and Installation 3

3. Desktop Interface Tour 5

4. Models, KV Cache, Context, and Thinking 9

5. Sending Prompts, Steering, Stop, and Progress 11

6. Work Modes 13

7. Projects and Workspaces 14

8. Chat Management 15

9. Context, Compression, and Pinned Files 16

10. Three-Layer Memory 17

11. Optional Skills 17

12. Internet and Research 18

13. Writing and Document Export 19

14. Development Workflow 19

15. Computer Control 22

16. Autonomy and Confirmations 23

17. Terminal UI and CLI 23

18. Files, Storage, Backup, and Logs 24

19. Troubleshooting 25

20. User-Facing Tool Reference 27

21. Practical Request Examples 29

22. Operational Notes and Limitations 29

23. Extended Workspace Operations 30

1. What Marvin Is

Marvin is a private Windows desktop application for running local language models as a general chat assistant, research assistant, writing partner, coding agent, and computer operator. Model inference runs on your workstation through `llama.cpp`; conversations and project files remain on local storage.

The application is designed for one user and one active model. It does not use parallel model agents. The active model can, however, run independent read-only tools in parallel and can leave long commands running in the background.

NOTE: Model inference is local. When the model uses `web_search` or `web_fetch`, the application makes ordinary internet requests to search engines and public web pages. Disable web access in `config.yaml` or avoid Research/Web requests when you require a fully offline session.

Main capabilities

- General conversation without coding behavior.
- Multi-source web and document research with a final synthesis.
- Writing and revision of scripts, articles, reports, treatments, and other documents.
- Local project development with file, Git, shell, test, image, and rollback tools.
- Screenshot-based control of Windows applications.
- Persistent projects, multiple chats per project, three-layer memory, pinned files, and optional skills.
- English and Czech interface, switchable without losing the active chat.

What the model can see

Selecting a project gives the model access to that directory through tools. It does not place every file body in the context window. The model receives a compact project map and reads individual files when useful. Pinned files and the three memory layers are exceptions: they are deliberately included as persistent context.

2. Requirements and Installation

Recommended workstation

Component	Recommended configuration
Operating system	Windows 11, 64-bit
GPU	NVIDIA RTX 5090 with 32 GB VRAM
Driver	Current NVIDIA driver compatible with the bundled CUDA build
System RAM	Enough for Windows, model mapping, projects, and tools; 64 GB or more is comfortable
Free disk space	Depends on selected models; Flash-Next IQ3_S needs about 92 GB (84.5 GB of weights and about 7 GB it prepares)
Python	Full: private Python 3.12 included. Minimal: install 64-bit Python 3.12 separately
Desktop window	WebView2 prepared automatically; Full includes the offline runtime
Browser tools	Microsoft Edge, normally present on Windows 11

Other NVIDIA GPUs may work, but the supplied contexts and quantizations were tuned and tested for a 32 GB RTX 5090. Lower-VRAM cards require smaller contexts, lower quantization, fewer GPU layers, or CPU offload.

NOTE: There are two installers: **Minimal** downloads packages and llama.cpp and requires separately installed Python 3.12. **Full** includes its own Python 3.12, locked packages and llama.cpp/CUDA libraries. Full creates a new isolated venv locally and does not register Python or change the system PATH. Both download models separately; the NVIDIA driver is installed by the user. Offline Backup remains a separate package.

WebView2 is detected and installed automatically during setup. Full includes the complete runtime; Minimal downloads it only when needed. An existing compatible runtime is reused. If WebView2 is removed later, Marvin prepares it again at startup. You do not need to visit Microsoft websites or install it separately.

Installing with Setup.exe

1. Choose Full for a self-contained application runtime. For Minimal only, install 64-bit Python 3.12 and enable **Add Python to PATH** first.
2. Run `Marvin-Setup-<version>.exe`.
3. Choose English or Czech in the installer. English is the default.
4. Choose whether to create a desktop shortcut.
5. Keep the post-install environment/model setup selected on the first installation.
6. Allow the console setup to complete. It creates the Python virtual environment, installs dependencies, downloads `llama.cpp`, and downloads the configured models.
7. Start **Marvin** from the Start Menu or desktop.

The default installation directory is:

```
%LOCALAPPDATA%\QwenHarness
```

Download size depends on the selected models. Interrupted downloads can be resumed by running **Set up environment and models** again from the Start Menu. The optional Flash-Next model downloads when selected in the application or in the installer's model list.

Qwen3.8-Flash-Next

Select **Qwen 3.8 Flash-Next · IQ3_S** in model settings, or **Qwen 3.8 Flash-Next · IQ2_XS** for smaller weights that answer faster on PCs with 48 or 32 GB of RAM. On the first selection Marvin prepares everything the model needs: its model engine, the weights (downloaded and verified, including image support), the model files it reads the weights through, and a draft layer of about 5 GB that speeds up answers. Each phase shows its progress; a transfer can be stopped and resumed later. On this PC the whole first preparation took about 20 minutes, most of it the draft layer's download; later starts take 30 seconds to a minute and a half.

Chat, attachments and tools use the same controls as with any other model, and you can switch to Flash-Next and back within the same chat: the conversation, its files and earlier answers carry over.

Flash-Next keeps most of its weights in system RAM and the parts it uses most on the graphics card. It runs on NVIDIA cards with 16 GB or more, on PCs with 32 GB of RAM or more, with NVIDIA driver 580 or newer. Marvin picks the profile measured for your card and your RAM; both offer 256k and 128k of context. On a 48 or 96 GB card it uses the same settings as on a 32 GB card and fills the extra graphics memory with more of the model's experts, which makes it faster (estimated, not measured). A long new document is read in about a minute; follow-up questions reuse what was read.

First launch

Marvin automatically starts the last successfully used model with its KV cache profile. This also applies when starting the web interface directly. A failed model switch does not replace the last successful selection. Start/restart buttons light up immediately and gently pulse during loading; the model panel shows the current phase, elapsed time, and GPU memory usage. Stop remains available during loading.

File uploads, exports, settings changes, restores, and other requests display immediate progress feedback followed by confirmation or an error. Confirmation means the request was accepted; queued model work continues in the conversation's progress display. Backup operations show live output and a completed/failed result in Data and backups.

The desktop launcher checks the Python environment, dependencies, `llama.cpp`, and the required model files. Missing components trigger the setup workflow. Once installed, ordinary launches do not download the models again.

Starting the desktop application opens the native WebView window, starts the Web UI, and starts the selected `llama-server` model when required. Closing the desktop application stops its Web UI and model server and releases GPU memory.

Updating

Install the new Setup.exe over the existing installation. User-created data, models, sessions, projects, user skills, and saved UI state are preserved. Changed application files and bundled system skills are updated. Models are downloaded again only when a configured model file is missing.

Uninstalling and removing all data

Windows uninstall removes installed application files. Large runtime data and user-generated data may remain so that an update or reinstall does not require downloading everything again.

To remove everything after uninstalling, inspect and delete the remaining installation directory only when you no longer need:

- `runtime\models` - GGUF model files.
- `sessions` - chat history, attachments, research ledgers, and exports without a project.
- `projects` - projects created inside the application.
- `memory` - global and work-mode memory.
- `user-skills` - personal skills.
- `projects.json` - project registry.

WARNING: Deleting these directories is permanent. Back up projects, sessions, memory, and user skills first.

Running from the source checkout

```
py -3.12 -m venv .venv
.venv\Scripts\python scripts\setup_env.py --model auto
npm --prefix frontend ci
npm --prefix frontend run build
.venv\Scripts\python launcher\launcher_app.py
```

For terminal use, run `run_cli.bat` or `.venv\Scripts\python tui.py`.

Source development requires Node.js to build the frontend. A normal Setup.exe installation already contains the built frontend and does not require Node.js.

3. Desktop Interface Tour

The workspace has project/chat navigation on the left, the conversation in the center, and a closable detail panel on the right. The prompt stays at the bottom of the conversation. At smaller widths, the detail panel becomes a drawer.

This chapter describes the 1.8.0 workspace. The top bar contains the model and its status. Open the right column with the detail icon above the conversation or the context indicator below the composer. Its content scrolls independently; the bottom **© Petr Sajner 2026** footer stays in place. The same copyright appears during startup.

Your first task, step by step

1. Click the model name in the top bar and check **Model and device**. Normally leave memory detection on automatic and wait for **Ready**.
2. Select a project, attach an existing folder through the project menu, or keep **No project**. Click **New chat**.

3. Choose the work mode beside the chat title, such as **Writing** for a document or **Development** for a repository.
4. Describe the result you need. Add source material with **Attach**, drag and drop, or clipboard paste when useful.
5. Send the message. Follow the conversation and the right-hand **Progress** tab. During work, you can clarify the request, queue another message, or stop the task.
6. Find created files under **Results**, open a preview or their folder, and verify the result. Ordinary text answers remain in the conversation.

Navigation and work modes

The navigation and detail columns can be resized by dragging their inner edges. Double-click an edge to reset its width; restarting or reloading the interface also restores the default widths. The conversation list initially shows the 20 most recent chats in the selected project. **Show older** reveals another 20; **Show recent only** collapses the list.

The desktop launcher uses the data folder belonging to its own installation. Running a development checkout alongside an installed copy does not merge their conversations. For explicit access to an existing installation from source, run `python marvin_web.py --data-dir "C:\path\to\Marvin"`; this uses the existing data in place without converting or copying it.

The project selector opens the most recent chat of that project. **No project** opens an independent chat. Selecting another chat never rebinds the task that is already running elsewhere.

Work modes always appear in this order: **Discussion, Research, Writing, Development, Computer**. The mode belongs to the chat. A running request retains the configuration with which it started.

Use **New chat** to create a conversation. Type into the title above the chat and press Enter to rename it. The chat's three-dot menu contains Move, Undo last turn, Branch, Export, and Delete. Selecting a destination moves the chat and updates its project context. Stop the chat's task before moving or deleting it.

The three-dot menu beside the project selector creates a new project, attaches an existing folder, or removes the selected project. Attached folders are never deleted from disk. Search in the left sidebar searches saved conversations.

Finding anything you have written

The box at the top of the sidebar searches **everything**, not only conversations: your chats, the text files in the open project, all three layers of memory, and the project's decisions. Results are grouped by where they live, and clicking one opens that place - the conversation, the document, the memory page, the decisions panel.

Which project is searched follows the open conversation. With no project open, only conversations and global memory are searched, because there is nothing else to look in.

Go to - one keystroke

Ctrl+K opens a small box you can type into. It offers the things the interface can do - new chat, switch work mode, the capability catalogue, the context, progress and changes panels, each settings page, dictation when it is ready - and below them, whatever your words found in your own work. Enter runs the first action; Escape closes it.

It is a shortcut, not a separate feature: everything it offers is somewhere in the interface already.

What can I ask for?

Beside **Attach** is **What can I ask for?**. It opens a catalogue of about twenty things Marvin can do, written as requests rather than as a list of functions, each with a ready example. **Use this** puts the example into the prompt, where it can be edited before sending; nothing is sent by pressing it.

The catalogue shows what the current work mode offers. An entry belonging to a different mode is shown in grey with the mode it needs and a button that switches the conversation to it - proper research, for instance, is a Research-mode capability, not a way of phrasing a question. **Show everything** lists all of them regardless of mode. Where a capability already has its own place in the interface - memory, decisions, task changes, research sources - the entry links there instead of repeating it.

Prompt, Attach, and image previews

The visible **Attach** button is always below the prompt. It opens the disk file picker and accepts multiple files. Images can also be dragged into the workspace or pasted from the clipboard with Ctrl+V. Ordinary text pasting still works.

Every pending image has a real thumbnail and an individual remove button. Removing an image leaves the prompt and other attachments intact. Clicking a thumbnail enlarges the original. After sending, thumbnails belong to the user message and remain available after reopening the chat.

PDF, DOCX, XLSX, CSV, Markdown, and text files can also be attached. The model receives their file paths and uses the document tools to read them. A text-only model cannot inspect image content; choose a vision-capable model when required. Draft text and uploaded attachments are retained when a request is rejected.

Enter and Ctrl+Enter send; Shift+Enter inserts a new line. The composer clears after the service accepts the identified message. Draft text is stored per conversation.

Suggested next step

While a conversation goes on, the model keeps forming its idea of what you will most likely ask next: what you are working toward, what has been solved and what is still open. When the chat is idle and the composer is empty, that idea appears in it as grey text. Click in the composer and press the right arrow (→) or Tab to turn it into real text, then edit it or send it as it is. Typing anything else replaces it. When the model offers no idea, a finished task suggests what its own state shows is still open: a failing check, an unfinished step of the task plan, or the first of the summary's next steps. The suggestion is part of the model's ordinary replies, so it costs no extra request and does not slow anything down. **Settings > Behavior > Suggest the next step** turns it off; the model then stops forming it, and an existing conversation reads its prompt once more on its next request.

Dictation

Turn **Voice input** on in **Settings > Behavior**. The first time, about 556 MB is downloaded in the background: the speech program and a Czech-capable model. Everything then runs on the processor, on this computer, with nothing sent anywhere.

A **Dictate** button appears beside Attach. Press it, speak, press it again. The text is written into the prompt box, where you read it and correct it before sending. Nothing is ever sent by voice alone.

Say which language you speak under **Dictation language**; leaving it on the interface language is normal. Stating it is roughly twice as fast as letting the program work it out, because detection is a separate pass over the recording.

What to expect: ordinary Czech comes back clean. Names, foreign words and technical terms are where mistakes appear, which is why the text lands in an editable box rather than being sent. Transcribing takes about four seconds, regardless of whether you spoke for two seconds or ten.

If you press the button and then say nothing, you get "Nothing was heard" rather than a sentence the program invented - silence is detected before transcription starts.

Image generation

Everything else in Marvin runs on your own computer. This one feature does not: it generates pictures through OpenArt, on your OpenArt account, and each picture costs credits. It is off until you turn it on.

Turn **Allow paid image generation** on in **Settings > Behavior**. A 4 MB program is downloaded once. Then press **Sign in to OpenArt**: the sign-in opens in your browser and you complete it there. Marvin never sees your password, and never stores any credential - the OpenArt program keeps its own, in your user profile, and nothing about it reaches Marvin's settings file, an export or a backup. There is no API key to paste, because OpenArt does not use one.

Once the account is connected it is shown as a highlighted label with your credit balance. **The switch is separate from being signed in**: turning it off stops image generation even while the account stays connected.

The model can then generate a picture when one is actually wanted - a sprite, a texture, a mock-up - and it saves into `generated-images` inside the current project, where you can open it like any other file. It knows five models: Google Nano Banana 2 (the default, 4K with accurate text in the picture), OpenAI GPT Image 2.5 in a faster and a

more precise variant, Nano Banana Pro for long text and consistent characters, and Seedream 4.5 for anime and illustration.

What to expect: about seventeen seconds for one picture, and the price is quoted in the answer beside the file. A picture from Nano Banana 2 cost 20 credits when this was written. Marvin does not cap the spending; the switch is the control.

Without an internet connection this is the one thing that cannot work. It says so and nothing else is affected.

Running tasks and the message queue

While a task is active, choose **Clarify now** to steer it or **After completion** to queue a separate request. A task in a different chat keeps running; a new request waits for the single model worker. Queued text can be edited, and queued messages can be cancelled.

Stop task works during prompt processing, reasoning, visible text, tool preparation, and synchronous actions. It may finish the current visible sentence briefly; it does not wait for a whole reasoning block. Already completed work and visible partial text are retained. Stop also pauses the pending queue; **Resume queued messages** starts it again. **Continue** resumes the interrupted task using the currently selected model and its current KV profile. It does not reload the original model after you switch to another one. Existing history and task progress remain available; a smaller context uses the usual history compression.

Refreshing or reconnecting the browser does not own or cancel the model run. After a full application restart, interrupted work is available to continue. If a tool result was not saved before a crash, the model is told to inspect the actual outcome instead of blindly replaying the action.

Results, Progress, and Context

Results lists created documents and changed files, actual recorded checks, research exports, and restore points. Open a file to preview it, open its folder, or download it. HTML results can be previewed as a local application; supported programs also offer **Run**.

Progress shows the task plan, durable operational notices, changed files, background processes and their output, and the isolated browser state. Background processes are stopped separately when appropriate.

Context shows the estimated current input size and physical context limit, measured usage from the last completed model request when available, three active memory layers, pinned files, and loaded skills. Pin or unpin individual files, clear pins, compress earlier context, or hand off to a new conversation.

During generation, the activity line distinguishes preparation, thinking, writing, tool-call preparation, and execution. It includes elapsed time and an explicitly approximate live token count.

Settings

Click the settings icon or the model status in the top bar. Settings are grouped into:

- **Model and device**: model, KV profile, vision capability, GPU memory choice, runtime diagnostics, Start / Stop / Restart.
- **Behavior**: autonomy and default handling of messages sent during work.
- **Memory and skills**: edit global, mode, and project memory; read or use a skill; design a skill; open user and project skill folders.
- **Data and backups**: choose where new projects are created, export/import a complete project, import chat JSONL, create/select/verify the local runtime backup, and monitor maintenance operations.
- **Appearance and language**: dark, light, or system appearance; spacing; English or Czech.
- **Help and manuals**: both PDF manuals and the slash command reference.

Thinking depth also stays beside the prompt because it can change between questions. Model settings selected while a request runs apply to subsequent requests. Language and theme changes update the interface without discarding the conversation.

Project decisions

Project decisions holds proposed, accepted, and retired decisions. Add or edit a decision, choose its status, and open its originating conversation. Only accepted decisions are included automatically in project context. They supplement the three memory layers; a proposal is not silently treated as an accepted fact.

4. Models, KV Cache, Context, and Thinking

Installed model profiles

GPU class	Model	KV precision	Context choices
16 GB	Qwen IQ3	Q8	64k / 48k
24 GB	Qwen IQ3	Q8	128k / 96k
24 GB	Qwen Q4	Q8	96k / 64k
24 GB	Qwen Q5	Q8	96k / 64k
24 GB	Nemotron Q4	Q8	512k / 256k
24 GB	Nemotron Q5	Q8	512k / 256k
32 GB	Qwen Q4	Q8	256k / 192k
32 GB	Qwen Q4	F16	128k / 96k
32 GB	Qwen Q5	Q8	192k / 128k
32 GB	Qwen Q5	F16	128k / 96k
32 GB	Ornith Q5	Q8	256k / 192k
32 GB	Nemotron Q4	Q8	512k / 256k
32 GB	Nemotron Q5	Q8	512k / 256k

On 32 GB-class cards, the new-installation default is Qwen Q5 with Q8 KV and 192k context. Smaller cards select an appropriate Q8 profile from the table. F16 is an explicit option only for Qwen Q4/Q5 on 32 GB-class cards. IQ3 is offered on 16/24 GB cards only. Each model and KV precision group offers the two highest relevant context choices.

Cards with 48 GB or more (64 and 72 GB cards included) use their own **48 GB** profiles and cards with 96 GB the **96 GB** ones. They are estimated from the 32 GB measurements, not measured, because no such card was available: everything that fits a 32 GB card, plus Qwen Q4/Q5 at Q8 256k (also with MTP) and F16 256k, and Nemotron Q5 at 512k entirely on the GPU. Qwen and Ornith stay at most at 256k, Nemotron at 512k.

Qwen Q4/Q5 also offer opt-in **MTP variants** of their Q8 profiles (for example "Q8 · 192k · MTP" next to "Q8 · 192k"). An MTP variant generates about 2–3× faster using a small draft model trained for the same model; answer quality stays the same. It needs about 2–2.9 GB of extra GPU memory, and the 1.4 GB draft model is downloaded and verified automatically on the first start. On 24 GB-class cards only Qwen Q4 has MTP variants. If memory runs out during an MTP session, Marvin first switches to the same context without MTP, then tries the next lower context with MTP, and finally that context without MTP.

While a speculative profile is running, the header shows an **MTP** badge next to the model name. The generation status line under the conversation shows the live token rate (tokens per second) together with the token estimate, so the speed difference between plain and MTP profiles is visible while you work.

Semantic search (Settings > Model and device) adds meaning-based retrieval over the project files and past conversations — Czech and English, including paraphrases — combined with keyword search. It is off by default; enabling it downloads a small embedding model (~635 MB) that runs only on the CPU, without touching the graphics card. The first search in a project builds the index in the background; the agent sees the progress and repeats the search when needed.

Nemotron is text-only. Qwen, Ornith, and Flash-Next use their own image support. The model list reports file availability; Flash-Next requires its shards, its projector and its prepared files, and Marvin prepares any that are

missing when you select it. Qwen also offers compact profiles for smaller cards. The table does not promise that every combination will run on every PC; the selected model's settings show its exact profile choices.

GPU memory budget and automatic recovery

Normally leave **Settings > Model and device > GPU memory budget** on **Automatic detection**. A manual value limits the memory Marvin may use; it cannot increase the physical card's capacity. Change it when no task is running. Marvin releases the previous model, selects a compatible profile, and restarts the model when required. It remembers the successful model and context for each budget, so returning to Automatic restores that configuration. If a change fails, it attempts to restore the previous working model and settings.

Qwen IQ3 uses Q8 64k/48k on 16 GB, and Qwen Q5 uses Q8 96k/64k on 24 GB. Both retain vision with CPU image processing. Nemotron Q4/Q5 offer 512k/256k on 24 GB by executing some experts on the CPU. The 32 GB Nemotron Q5 512k profile also executes one expert block on the CPU; 256k uses GPU weights. These placements were measured on an RTX 5090, not physical 16/24 GB cards. Other programs can affect whether the larger choice starts successfully.

If an actual allocation failure or sustained critical physical RAM interrupts a task, Marvin retries the next smaller context with the same model, cache precision and weight placement. Completed tool results remain in history. It never alternates Q8 and F16 during recovery. A session that started on an MTP variant first drops the draft at the same context, then interleaves lower contexts with and without MTP. If no smaller supported context remains, the task stops with a memory explanation; select another model and use **Continue**. Successful settings are remembered for each GPU budget.

Flash-Next settings and realistic waiting times

IQ3_S downloads 84.5 GB in three files. **IQ2_XS** downloads 69 GB, of which 29.7 GB (the second shard and the image projector) are the same files as IQ3_S's, so beside IQ3_S it adds about 39 GB. The first preparation adds about 7 GB of model files and 0.7 GB for the model engine. Later starts use the local copies; changing the context repeats none of it.

Loading takes about 30 seconds with 64 GB of RAM and up to about a minute and a half on smaller PCs, which read part of the weights from the SSD. While the model fills system RAM the PC may respond slowly for a moment.

Both weights use Q8 cache with 256k and 128k profiles. Marvin chooses by your card (32, 24 or 16 GB) and your RAM (64, 48 or 32 GB; more than 64 GB uses the 64 GB profiles). Measured in October 2026 on an RTX 5090, with smaller cards and RAM simulated; *answer* is a short reply with thinking off, *long input* the first answer to an input that fills 256k:

IQ3_S	32 GB card	24 GB card	16 GB card
64 GB RAM	110 tokens/s, long input 43 s	92 tokens/s, 43 s	73 tokens/s, 49 s
48 GB RAM	72 tokens/s, 56 s	41 tokens/s, 71 s	25 tokens/s, 4.3 min
32 GB RAM	45 tokens/s, 2.9 min	24 tokens/s, 4.7 min	13 tokens/s, 10 min

IQ2_XS answers at 107–136 tokens/s with 64 or 48 GB of RAM on every card, and at 119 / 71 / 32 tokens/s with 32 GB of RAM on a 32 / 24 / 16 GB card. A cached follow-up returns in about one second everywhere. For comparison, the previous Flash-Next build answered at about 25 tokens/s and needed about 44 minutes for the same long input on the strongest PC.

If memory runs short during a task, Marvin restarts the same model at 128k with the same settings and continues; completed tool results stay in the history. With 48 GB of RAM and a 16 GB card, a very long input can exceed the memory even at 128k; then select a smaller model and use **Continue**.

Profile names 128k/192k/256k refer to 131,072 / 196,608 / 262,144 tokens. The numeric indicator may round the same capacity to a value such as 262k.

Choosing Qwen Q5 or Q4

Use Qwen Q5 for serious development, architecture, difficult reasoning, and final-quality writing. Use Qwen Q4 when speed or the full 256k context matters more than the small quality advantage of Q5.

Q5 with Q8 is a strong general default. On 32 GB-class cards, the optional F16 choices are 128k and 96k.

Ornith

Ornith is a Mixture-of-Experts model and generates very quickly because only part of its weights is active per token. All model weights must still fit in VRAM. In practical application development it may produce weaker results than dense Qwen despite its speed.

Ornith supports thinking on/off natively. The intermediate depths are prompt-guided rather than native `reasoning_effort` values:

- `xhigh` adds deliberate planning, architecture, edge-case, and verification guidance.
- `medium` adds moderate reasoning guidance.
- `low` uses Ornith's ordinary thinking behavior.
- `off` disables thinking.

KV cache precision

KV cache stores the model's attention history. It is separate from weight quantization: **IQ3_S** describes Flash-Next's weights, while **Q8** describes its context cache. Changing KV does not change the downloaded weight files.

Setting	Benefit	Tradeoff
F16	Higher KV precision	Uses more memory for the same context
Q8	Lower cache memory use	Some cache precision is exchanged for memory savings; capacity depends on the profile

Changing KV precision restarts the server. It does not delete or reset the chat.

Thinking levels

For Qwen, `xhigh`, `medium`, and `low` are sent through the model's native reasoning-effort control. `off` disables the thinking block. The setting can be changed between prompts in the same chat.

Reasoning tokens consume generation time and temporary context while the response is being produced. The UI shows a live estimate. Reasoning is saved with the answer and can be expanded in history. Persistent context estimates include reasoning sent back to the model; measured usage refers to the last completed request.

No artificial task or output limit

The production agent has no step limit and no application-level output-token cap. It continues until the task is complete, the model reaches its physical context boundary, a tool/server fails, or the user presses Stop.

5. Sending Prompts, Steering, Stop, and Progress

Ordinary prompt

Write the request in natural language. The model automatically receives the active work mode, project, memory layers, current project snapshot, pinned files, and available skill metadata.

For best results, state the result you want, important constraints, and how completion should be verified. The model is allowed to choose its process unless your prompt specifies it.

Image attachment

Use **Attach**, drag and drop, or Ctrl+V to add one or more images. Supported formats are BMP, GIF, JPEG, PNG, and WebP. The most recent images remain in active context; older image references stay in history but may be omitted from later API requests.

Images can be photographs, diagrams, screenshots, UI references, error messages, or visual source material. Qwen and Ornith use their matching vision projector automatically.

Steering a running task

With **Clarify now** selected, another message in the running chat steers the active task:

1. The clarification is accepted immediately and the composer clears.
2. The current generation stops after the nearest completed sentence or safe chunk.
3. The finished partial response remains visible and saved.
4. The clarification is inserted into the same task.
5. The model resumes with the updated instruction.

Use steering for corrections such as "keep the existing camera behavior", "do not change the API", or "also export this as PDF".

Choose **After completion** for a separate next request. A message in another chat waits for the single model worker. A message arriving at the completion boundary is retained as queued work instead of being stranded.

Stop

Stop directly signals the independent run controller, including while no stream bytes arrive. It requests a graceful generation stop and normally allows the nearest sentence to finish. It also cancels the currently awaited browser operation or synchronous `run_command`, terminating that command's process tree. Finished partial text and captured command output are retained. A new prompt starts cleanly afterward.

Long-running operating-system processes have their own termination controls in **Progress > Processes**. Each is labelled **Stop this command** and ends only that background command; the task itself keeps running. Use **Stop task** beside the prompt to end the work.

Interrupting while the context is being read. A large prompt is read once and then reused, so each further step only adds the new part. Cutting that read short discards everything already read, and the next request pays for all of it again - on a very large context that is minutes of work. A clarification therefore waits for the read to finish and is applied to the answer straight afterwards; the conversation says so while it waits. **Stop** is not made to wait, because it is an explicit instruction - but after stopping during a context read, the next message starts that read again from the beginning.

Stop task beside the composer ends the current work and pauses the queue; the model can remain loaded. **stop** in **Model and device** stops the model server and releases its resources. Downloaded weights remain available for the next start.

When something fails

A failed task leaves a message in **Progress > Activity history** instead of only a notification that fades. It shows what failed and, where Marvin recognises the cause, what to do about it: a project without its own virtual environment, a model that did not start, no model fitting the memory budget, a project folder that was moved, or the machine running out of memory.

Where there is no known answer nothing is invented. **Work out what to do** puts a prepared question, including the error, into the prompt and hands it to the model, which can usually work the cause out. The question is not sent by itself, so it can be read and changed first.

The same offer appears on a failed tool result in the conversation and on a project check that failed, timed out or could not start.

Live progress

While the model reads the context, the line reports how much of the conversation the server still had from the previous request, as a share of the whole prompt, plus an estimate of the time left at the rate being observed.

A high share is the normal case: the harness only appends to a request, so the model re-reads nothing. A low share means something near the start of the conversation changed and all of it is being processed again. Two things do that on purpose - compression, and giving up older screenshots when the context fills. Both are reported in the chat when they happen.

The live chat distinguishes:

- Thinking/reasoning.
- Visible response text.
- Preparation of large tool arguments, such as code for `write_file`.
- Execution of tools, commands, tests, searches, and file operations.
- Seconds without new tokens.

For example, while a large program is being generated the UI can show that content for a named file is growing instead of appearing frozen.

6. Work Modes

Each chat stores its work mode. Changing to another chat restores that chat's mode and project context.

Discussion

Use for ordinary conversation, analysis, learning, brainstorming, planning, and non-programming questions. Discussion does not inject coding procedures. It can use memory, internet, project documents, skills, pinned files, and document export.

Research

Use for web research, literature or market investigation, comparison, and evidence synthesis. Research creates a persistent plan and ledger, keeps all sources regardless of perceived trustworthiness, records contradictions, and produces a final synthesis.

Watching a document being written

In the **Files changed** list, a document the task touched has a second button beside the difference view. It opens the document rendered, and **keeps it up to date while the model writes**: the view re-reads the file and redraws only when the content really changed, so it does not flicker.

The button appears for what can be rendered - Markdown, text, DOCX, PDF, HTML, CSV and spreadsheets. Source code and binary files have nothing to show, so they only offer the difference view.

This is the answer to the one question Writing mode could not answer while it worked: what does it look like now.

Writing

Use for scripts, articles, reports, treatments, notes, and document revision. Writing can read/write project files, use web and documents, view images, create checkpoints, and export Markdown/DOCX/PDF. It does not expose Git and shell tools unless you switch to Development.

Development

Use for programming and repository work. The model receives an automatic project map and can read/search/write files, apply atomic patches, use Git, run shell commands, start tests, inspect images, maintain a task checkpoint, and roll changes back.

Computer

Includes Development tools plus screenshot, mouse, keyboard, and scrolling controls. Use it when the model must interact with a GUI application rather than only project files.

NOTE: Work modes are capability profiles, not separate model instances. Switching mode does not load another model.

7. Projects and Workspaces

No project

Choose **No project** for conversations that should not access a project folder. Chats without a project appear in their own list. Exports are stored in the chat session directory.

Creating a managed project

1. Open the three-dot project menu.
2. Choose **New**.
3. Enter a project name and confirm.

The application creates a new folder under its `projects` directory, registers it, creates project memory when needed, and uses it as the workspace.

Where new projects are created

Settings > Data and backups chooses the folder for projects created by name. Until it is set they are created beside the installation. The folder has to exist, be writable, and lie outside the model runtime and the conversation history; anything else is refused with the reason.

Changing it moves nothing. Projects that already exist keep the folder they were created in and continue to work from there - only the next one goes to the new place.

Attaching an existing folder

Choose **Attach**, select an existing directory, and the application registers it as a project without copying its files. The folder itself remains in its original location.

Selecting a project

Select the project in the project dropdown. Relative file paths used by tools resolve inside this workspace. The project chat list updates to show chats whose metadata points to that project.

Multiple chats per project

A project can contain any number of chats. Each chat has independent history, work mode, context compression, pinned files, research ledger, and unfinished-task state. They share the physical project directory and project memory.

Moving a chat

In the chat's three-dot menu, open **Move to project** and select a target project or **No project**. The move happens immediately. The chat's workspace, project memory, document library, and model context are reconfigured to the target.

Deleting a project

1. Select the project.
2. Open the three-dot project menu.

3. Click **Delete project**.
4. Read the exact path shown.
5. Click again within eight seconds to confirm.

For a project created inside Marvin, deletion removes the project registry entry, every chat belonging to the project, the project folder and all files inside it. For a project attached from your own folder, deletion only removes the registry entry and the project's chats — **your folder and its contents always stay on disk**. If such a folder cannot be deleted for another reason, nothing is removed and a readable error is shown.

If a folder was removed outside the application, the project is marked missing. Reattach or remove the stale project entry as appropriate.

8. Chat Management

New chat

New chat creates a transient empty chat in the current project/mode. It is written to disk only after the first user message.

Delete chat

Click **Delete**, then click again within six seconds. The session folder, messages, attachments, research ledger, task state, and session-local exports are removed. Project files are not affected.

Rename

Type a title in the rename field and press Enter. The chat lists and search index update.

Retry

Retry keeps the last user prompt, removes its answer and any compression state created after it, and asks the model to answer again.

Undo

Undo removes the last complete user/assistant round from the chat. It does not roll back project files. Use **Revert task changes** for filesystem rollback.

Fork

Fork creates a new chat branch ending at the last user prompt. It copies relevant image attachments, project assignment, work mode, and pinned-file list. The original remains unchanged.

Search all chats

Open **Search all chats**, enter a word or phrase, and select a result. Search uses a persistent SQLite full-text index and covers saved chats across projects.

Export and import chat

The active-chat **Export / import** group supports:

- Markdown export for human reading.
- JSONL export preserving the message structure.
- JSONL import as a new chat.

Import never overwrites the original session. Its system prompt is refreshed to the current application version.

Handoff to a new chat

Use **Context > Hand off** for a long conversation that should continue in a fresh chat. The model summarizes essential goals, decisions, state, findings, and next steps; a new chat is created with that summary while the old history remains intact.

9. Context, Compression, and Pinned Files

Context indicator

The status and **Context** panel show estimated tokens, visible versus total messages, images, compression state, and pinned files. The detailed breakdown separates conversation/attachments, current project context, and tool definitions, so the displayed total includes tool-schema overhead.

Automatic project snapshot

In Development, the model receives a compact map of the current workspace, file types, key entry points, directories, and top-level Python symbols. It is assembled as a temporary tail for the current request and is not stored in chat history, so stale snapshots do not accumulate and the earlier reusable prefix remains unchanged.

Discussion, Research, and Writing use a project document catalog instead of a coding repository map.

The harness also discovers `AGENTS.md`, `QWEN.md`, and `CLAUDE.md` from the project root down to directories containing files currently being inspected or changed. Root guidance applies broadly; deeper files are more specific. These documents guide the model but do not override the current user request.

Pinning a file

Open **Context** and choose **Pin file**, or tell the model to pin a path. The complete text is refreshed into the context at the start of subsequent tasks in that chat.

Good pinned files include architecture decisions, an active specification, a handoff, or project rules that matter throughout the conversation.

Limits:

- Maximum 10 pinned files.
- Approximately 40,000 characters in total.
- Pinning belongs to one chat; a fork copies the pin list, a separate new chat does not.

Do not pin ordinary source files that the model can read on demand, large logs, or memory documents already injected automatically.

Manual and automatic compression

At approximately 85% of the selected model context, the harness first stops sending all but the four newest screenshots. In Computer mode they are by far the largest part of the conversation, so giving them up is usually enough and the text of the conversation is kept intact. The pictures stay visible in the chat; only the model stops receiving the older ones, and that decision is permanent for those messages.

If that is not enough, the harness summarizes older messages. The visible chat history is never deleted; only the model's request view changes to system prompt + summary + recent messages.

Use **Context > Compress** to compress earlier. If the model reports an overflow, the agent performs one automatic compression-and-retry cycle. A second overflow is reported instead of looping forever.

10. Three-Layer Memory

Every task receives up to three complete memory documents:

Layer	Scope	Typical content
Global	Every mode and project	User preferences, universal facts, stable conventions
Work mode	All chats using one mode	Research style, writing preferences, development conventions
Project	One workspace	Decisions, terminology, paths, project-specific facts

Locations:

```
memory\GLOBAL.md
memory\MEMORY.md                (Development mode legacy/default)
memory\modes\discussion.md
memory\modes\research.md
memory\modes\writing.md
memory\modes\computer.md
<project>\QWEN_MEMORY.md
```

Open the active memory files in **Settings > Memory and skills**. You can edit them directly or say, for example:

```
Remember globally that I prefer concise Czech summaries.
Remember for Research that every final answer must preserve contradictory findings.
Remember for this project that the delivery folder is D:\Exports.
```

The model chooses `global`, `mode`, or `project` storage through `save_memory`. Moving a chat to another project immediately changes its project-memory layer.

11. Optional Skills

Skills are reusable `SKILL.md` guidance packages. They do not run a second agent and do not override the user. Only each skill's name and trigger description are included in ordinary context; the body is loaded through `read_skill` when it clearly helps.

Skill precedence

1. Project skill: `<project>\.qwen-skills\<name>\SKILL.md`.
2. User skill: `user-skills\<name>\SKILL.md`.
3. Bundled system skill: `skills\<name>\SKILL.md`.

A higher layer with the same `name` replaces the lower one.

Bundled skills

Skill	Purpose
`systematic-debugging`	Reproduce, trace, test hypotheses, and fix the real cause.
`architecture-options`	Compare meaningful architecture options without overriding requested constraints.
`implementation-verification`	Proportionate requirement, artifact, test, and release verification.
`performance-investigation`	Measurement-first latency, throughput, memory, and bottleneck analysis.
`research-synthesis`	Complete synthesis that retains contradictions and all relevant sources.

Adding a user skill

Open **Settings > Memory and skills > User skills** and create:

```
user-skills\my-skill\SKILL.md
```

Minimum format:

```
---
name: my-skill
description: What this helps with and when the model should load it.
---

# My Skill

Flexible guidance, references, examples, or workflow suggestions.
```

The catalog refreshes automatically. Keep the description specific because it is the model's trigger. Put project-only skills in `.qwen-skills` inside that project.

12. Internet and Research

Ordinary web use

All work modes can use `web_search` and `web_fetch` for current information, public documentation, error messages, and web pages. `web_fetch` handles HTML and can extract text from supported PDF and DOCX downloads.

For long pages, the model can find a specific phrase or read the next passage without downloading the page again. The complete fetched source remains in the research record within its technical storage limit; a focused excerpt does not replace it. This is particularly useful for large models that perform part of their computation in system RAM.

During input processing, **Reading context** shows progress through new text and the number of tokens reused from cache. This is separate from generating reasoning and the answer. Reading a long document for the first time may be slow even when a follow-up question reuses nearly all prior context. These optimizations do not change reasoning effort or the chosen quantization.

The percentage on that line is labelled **new**, because it is progress through the part of the prompt the model server does not already hold - not how full the context is. How full the context is appears beside the composer, as `Context: ~132k / 197k`. That figure is an estimate between requests, and it corrects itself: the server reports the exact prompt length with every reply, so the estimate is measured against it and adjusted. A new conversation starts from measured averages and becomes more accurate as it goes, because Czech prose, program code and screenshots each cost a different number of tokens per character.

The default search backend is Google; `config.yaml` can select Google, Bing, or automatic fallback. Fetching is read-only HTTP/HTTPS.

Research workflow

Research mode automatically:

1. Records the question.
2. Creates a persistent research plan before searching.
3. Stores every search query and candidate link.
4. Stores the full readable content of fetched web/local sources.
5. Tracks coverage and source IDs.
6. Keeps contradictory and minority information.
7. Produces a final human-readable synthesis.

Sources are not hidden, discarded, or ranked because the model considers them untrustworthy. The user decides credibility and relevance. The synthesis distinguishes source claims from inference and states uncertainty and missing evidence.

Research panel and exports

Open **Research** in the right-hand **Results** tab. **All sources** opens the current task's source records; **PDF**, **DOCX**, and **Sources** export:

- Complete research ledger containing plans and source material.
- Completed synthesis as DOCX.
- Completed synthesis as PDF.

The general task plan and activity history are in **Progress**. Asking to export an already completed answer does not start a second research run.

Project documents

Readable project documents include Markdown, text, RST, CSV, JSON, YAML, PDF, and DOCX. Research mode records a local document as a ledger source when it is read.

13. Writing and Document Export

Writing mode keeps the user's intent, voice, structure, and constraints. It can read and revise project files without introducing development terminology unless the user asks for it.

The model can export final text in every work mode using natural requests such as:

```
Save the final answer as PDF.
Export this as a structured DOCX called production-report.
Create a Markdown file with this synthesis.
```

Supported formats:

- Markdown (`.md`).
- Structured Word document (`.docx`).
- PDF with Unicode/Czech text and basic tables/inline formatting.

Output location:

- With a project: `<project>\exports`.
- Without a project: `sessions\<chat-id>\exports`.

Exports produced through project file tools participate in the current task checkpoint and can be reverted when appropriate.

14. Development Workflow

Development mode is intended for complete repository work while keeping the main UI conversational.

Project discovery

At each request, the model receives a current repository snapshot plus applicable hierarchical project instructions. It can call `repo_overview`, `project_instructions`, list or glob files, search literal text or regex, navigate declarations with `find_symbol`/`document_symbols`, find whole-word uses with `find_references`, and read relevant ranges before editing.

File operations

- `read_file` reads text with line numbers.
- `search_files` uses ripgrep when available and supports literal/regex, case sensitivity, and filename glob.
- `find_files` lists matching project files through a glob.
- `write_file` creates or fully rewrites a file.
- `apply_patch` performs exact atomic text replacements.
- `make_directory`, `move_file`, and `delete_file` provide structured filesystem operations. Their changes participate in task rollback.
- `view_image` brings an image file into vision context.

Isolated browser workflow

Development and Computer modes include a persistent headless Microsoft Edge session. This is separate from the user's normal browser window and is intended for testing web applications:

1. `browser_open` loads a local or public URL.
2. `browser_snapshot` returns visible text and interactive elements with refs such as `e1`.
3. `browser_fill`, `browser_click`, and `browser_press` interact through those refs.
4. `browser_select`, `browser_hover`, and `browser_scroll` cover dropdowns, hover states, and long pages.
5. `browser_upload` sets a local file input; `browser_download` stores a downloaded file under the active chat.
6. `browser_viewport` switches between desktop, tablet, and mobile layouts.
7. A new snapshot verifies DOM-visible results; stale refs are rejected.
8. `browser_console` and `browser_network` expose console messages, HTTP responses, and failed requests.
9. `browser_screenshot` attaches the rendered page to the next Qwen request for native vision inspection.
10. `browser_close` or **Browser session > Close browser** releases the Edge process.

The browser remains available across individual agent steps. Use Computer mode instead when the task concerns a native desktop application or the entire Windows screen rather than a web page.

Task plan

For substantial work the model creates an operational plan with `set_task_plan` and updates steps with `update_task_step`. **Task progress** shows the goal, pending/in-progress/completed steps, validation result, and whether the final diff was reviewed. This task ledger is stored in `task-plan.json`, survives restart and compression, and contains operational state rather than private chain-of-thought.

Task checkpoint and rollback

The first write in a task starts a change journal. **Changes in this task** lists created/modified files. **Revert task changes** restores all journaled files to their pre-task state, including removing files created by the task.

This rollback is independent of chat Undo. Chat Undo changes conversation history; task rollback changes the filesystem.

Git

The model can inspect status and diff and create a local Git commit. `git_commit` stages the paths explicitly supplied by the model or, when omitted, only files recorded by the current task journal. It never pushes automatically. Ask explicitly when you want a commit or push.

Commands and tests

Short commands run synchronously with a timeout. Their complete stdout/stderr is stored in `sessions\<chat-id>\command-logs`; the model receives a head+tail preview so errors at the end are preserved. Stop cancels the active synchronous command and terminates its process tree. Long commands run in the background and return a process ID. The model can poll output, send stdin, or terminate the full process tree.

`project_validation_profile` lists detected test, lint, typecheck, and build commands. `start_project_check` starts the primary command or a named check:

- This harness: `tests/test_core.py`.
- Python: a `tests/` directory, or `test_*.py` modules in the project root. The

stdlib `unittest` runner is used by default; pytest only when the project configures it (`pytest.ini`, `conftest.py`, `[tool.pytest]`) and the interpreter can import it.

- Node: `npm test` or `npm run check`.
- Rust: `cargo test`.
- Go: `go test ./...`.
- .NET: `dotnet test`.

Projects can replace auto-detection with `.qwen/project.yaml`:

```
checks:
  - id: tests
    label: Full test suite
    command: npm test
    shell: powershell
    kind: test
    timeout: 900
    primary: true
```

Checks run with the project's own virtual environment (`.venv` or `venv`) when it has one, so they see the project's dependencies. Without one Marvin's interpreter runs them and a missing third-party import is reported as the failure reason. Lint and type checks are offered only when their tool is installed, so a detected check never fails merely by starting.

Project status. The **Progress** panel lists the detected checks for the current project before they have ever run, runs them all without the model with **Run project checks**, and keeps the outcome per project in `.qwen/check-status.json` so it survives a restart. Each row shows pass or fail, when it ran, and its output tail on hover. **Fix failures with agent** opens a Development chat in that project and asks the agent to run the checks, fix what fails and iterate without weakening tests. Both buttons say so plainly when a project has no detectable checks.

Automatic task commits. In the project dialog (three-dot menu beside the project selector), the switch **Commit each finished task automatically** makes every successfully completed Development or Computer task commit exactly its own changed files in the project repository — subject from the task goal, body from the final summary, trailer linking back to the chat. The switch is off by default and Marvin never pushes; the chat shows a notice with the commit hash.

Completed checks are recorded automatically in **Task progress**. Before a changed task finishes, the harness gives the model one non-blocking reminder when validation, plan steps, or final diff review are still missing. The model can perform the useful checks or explain why one is not relevant.

The final verification phase is guidance, not a restriction. Explicit user requirements about architecture, format, scope, or a single-file result take priority.

15. Computer Control

Computer mode adds GUI tools to the complete Development tool set.

Operation cycle

1. The model calls `screenshot`.
2. The screenshot is downscaled if necessary and attached to the conversation.
3. The model identifies controls in image-pixel coordinates.
4. It clicks, moves, scrolls, types, or presses keys.
5. It captures another screenshot to verify the result.

The harness maps screenshot coordinates back to the real display resolution.

Available GUI actions

- Capture primary-monitor screenshot.
- Get screen size and coordinate mapping.
- Move mouse for hover states.
- Left/right/middle click and double-click.
- Scroll up/down at an optional location.
- Type ASCII directly or paste Unicode/long text through the clipboard.
- List the open windows, and photograph one of them by name even while another

program covers it. This is how the model watches a program it started without taking the screen away from you: a plain screenshot shows whatever is in front, which is usually not the program being tested.

- Send keys straight into one program by window title, without bringing it

forward. Measured against a covered game window: presses, releases and combinations such as Ctrl+S all arrive, while the window you are working in receives nothing. This is how the model drives a program it started while you carry on with something else.

- Bring a window to the front by name, restoring it if it was minimised. This is

the fallback for programs that read the keyboard directly rather than through window messages and can only be reached while in front - it takes the screen from you, so the model tries the path above first. If the system refuses to change the foreground, the model is told so instead of reporting success.

- Run an SDL or pygame program with no window at all, when the question is about

behaviour rather than appearance: it renders into memory, saves a frame as a picture the model can look at, and never touches your screen.

- Press keys and combinations such as Enter, Escape, Ctrl+S, Alt+F4, or Win+D.

Keys go out as hardware scancodes, which games and other SDL or DirectInput windows require: they identify keys by scancode and ignore a key sent without one, which is why such a window used to react to nothing at all. A key is also held down briefly so it cannot pass between two of the window's checks. If a game still does not respond, ask for a longer hold; the older delivery path stays available for the rare window that prefers it. The upper-left-corner failsafe applies to both.

Failsafe

Move the physical mouse pointer rapidly to the top-left corner of the primary display to trigger PyAutoGUI's failsafe and interrupt GUI actions.

WARNING: Computer mode can see the primary screen and can perform destructive actions. On-screen text can contain misleading instructions. Use Supervised autonomy for email, payments, account changes, deletion, or any sensitive workflow.

16. Autonomy and Confirmations

Autonomy controls confirmation of WRITE-risk tools. It does not change model intelligence or available context.

Level	Behavior
Supervised	Every write, shell action, and GUI action asks for confirmation. Read-only operations proceed.
Semi	The first WRITE action in a task asks; after approval the rest of that task proceeds.
Auto	WRITE actions proceed without confirmation.

There is no task-step limit in any production autonomy mode.

When confirmation is required, the chat lists the pending actions and shows **Allow** and **Deny**. Denying returns the decision to the model so it can adapt. Repeated clicks are ignored once the pending action is gone.

Read-only commands are classified conservatively. Redirection, package installation, arbitrary Python, copying, deletion, network writes, Git commit/push, and mixed command chains are treated as writes.

17. Terminal UI and CLI

The installer creates **Marvin (CLI)** in the Start Menu. It opens `run_cli.bat`, which launches the interactive terminal UI using the installed Python environment.

Commands

Command	Purpose
`/memory`	Show global, active-mode, and project memory.
`/model q4|q5|ornith_q5`	Switch model; restarts the server.
`/work discussion|research|writing|development|computer`	Select work mode.
`/mode chat|agent|computer`	Compatibility shortcut for legacy agent mode.
`/autonomy supervised|semi|auto`	Set confirmation behavior.
`/thinking xhigh|medium|low|off`	Set reasoning depth.
`/ws [path]`	Show or set workspace.
`/img <path>`	Attach an image to the next prompt.
`/screenshot`	Capture and attach the screen.
`/new`	Create a new session.
`/sessions`	List sessions.
`/load <id>`	Load a session.
`/server status|start|stop`	Control the inference server.
`/help`	Show command help.
`/exit`	Exit the CLI.

During confirmation, `y` allows, `n` denies, and `a` allows all remaining writes in the current task. Ctrl+C interrupts generation.

18. Files, Storage, Backup, and Logs

Important locations

Path	Contents
`runtime\models`	GGUF models and vision projectors
`runtime\llama`	`llama.cpp` CUDA binaries
`runtime\workspace-settings.json`	Current workspace UI preferences and the last successful model
`runtime\application.sqlite3`	Durable task queue, events, and file registry
`runtime\models\strata`	Flash-Next's weights and the model files prepared from them
`runtime\strata`	Flash-Next's model engine, prepared automatically
`sessions\<id>`	Messages, metadata, attachments, task state, research, compression, exports
`projects`	Managed project folders created by the app
`projects.json`	Registered project list
`memory`	Global and work-mode memory
`<project>\QWEN_MEMORY.md`	Project memory
`skills`	Bundled system skills
`user-skills`	Personal skills preserved by the installer
`<project>\.qwen-skills`	Project skills

Backup recommendations

Back up at least:

- Project directories.
- `sessions` when chat/research history matters.
- `memory`.
- `user-skills`.
- `projects.json`.
- State files in `runtime`, including workspace preferences and the application database.

The older `webui-state.json` supports compatibility and migration of earlier preferences; the main workspace uses `workspace-settings.json`.

The application can also create a complete reusable installation backup from files that are already present on this computer. Open **Settings > Data and backups** and select **Create backup**. Choose a parent directory on another disk when possible. The application creates a timestamped `QwenHarness-Offline-Backup-*` folder containing:

- Every complete model and vision-projector file currently in `runtime\models`.
- The installed `llama.cpp` and CUDA runtime from `runtime\llama`.
- The current model selection and `requirements.txt`.
- A snapshot of the Python packages already installed in `.venv`.
- The matching Setup.exe when available in the build directory; version 1.8 prefers Full with private Python.
- `manifest.json` with the size and SHA-256 hash of every backed-up file.

Backup creation copies model, runtime, and Python dependency files directly from the current Marvin directory. It does not download any of them again. It needs free space approximately equal to the installed runtime and may take time because every copied file is hashed.

Installing from the offline backup

1. Copy the entire offline folder, including `manifest.json`, `payload`, `python-dependencies`, and the included installer. The complete bundle with all current models requires over 200 GB of storage.
2. Run the included **Full** installer. It includes private Python and needs no separate Python installation. Only when using **Minimal**, prepare 64-bit Python 3.12 first.
3. Use either of these methods:

- When Setup.exe is inside the backup folder beside `manifest.json`, run it there; it detects the backup automatically.
- Put the backup beside Setup.exe and rename it exactly to `QwenHarness-Offline-Backup`; the installer detects it automatically.
- Install Marvin, open the Start Menu, run **Set up from offline backup**, and select the backup folder.

4. When Setup recognizes a backup beside itself, the first preparation restores local files first. A complete restore copies every model included in the bundle, including Flash-Next and its model engine, and verifies SHA-256. The wizard's model selection governs any additional downloads.
5. The Start Menu command **Set up from offline backup** also restores locally first. Ordinary setup without that selection uses the internet and a registered backup as fallback. Anything absent from the local bundle still has to be obtained separately.

Use **Use as fallback** to register a backup for future download failures, **Verify SHA-256** to check every file against its manifest, and **Clear selection** to stop using that fallback. Selecting a backup does not disable internet access. Keep Setup.exe, `manifest.json`, `README-OFFLINE.txt`, `requirements.txt`, `python-dependencies`, and `payload` together in the same backup folder.

The offline installation backup does not include chats, projects, memory, or personal skills. Back up those irreplaceable user files separately as listed above.

Logs

Log	Use
`runtime\launcher.log`	Desktop launcher and startup problems
`runtime\app.log`	Native app lifecycle/crash details
`runtime\webapp.log`	Web UI startup and Python errors
`runtime\llama-server.log`	Model loading, context, CUDA, inference, and timing
`runtime\strata-server.log`	Flash-Next loading and inference

19. Troubleshooting

Server does not start

- Open **Settings > Model and device**, read the error, and try restart.
- Confirm all selected model files are complete, including Flash-Next's shards and projector.
- Read `runtime\llama-server.log`, or `runtime\strata-server.log` for Flash-Next.
- Check NVIDIA driver/GPU availability.
- Stop another process using port 8080.
- If memory allocation fails, check both free VRAM and system RAM. Flash-Next keeps its weights in system RAM; other large programs left open reduce what it can use.
- Flash-Next needs NVIDIA driver 580 or newer.

Web UI port is occupied

The launcher searches for an available nearby Web UI port. If another Marvin Web UI already owns the configured port, the launcher reuses it. See `runtime\launcher.log` for the selected URL.

Model or projector is missing

Run **Set up environment and models** from the Start Menu. Existing complete files are skipped. The downloader resumes supported incomplete downloads.

Model is slow after a long conversation

- Check context usage.
- Keep Qwen Q5/Q8 for 192k or switch Q4/Q8 for 256k.
- Use manual compression or Hand off.
- Avoid repeatedly changing early persistent memory during one long task because a changed system prefix must be evaluated again.

Context overflow

The agent automatically compresses and retries once. If it overflows again, manually compress, hand off to a fresh chat, choose a larger context profile, or remove unnecessary pinned files/images.

Interface appears frozen

Read the live activity line. The model may be thinking, generating a large tool call, writing a file, running a test, or waiting for a long process. If there is no server activity for an extended period, the harness detects a stale stream and reports it. Use Stop when appropriate.

Isolated browser does not start

- Microsoft Edge must be installed in its standard Windows location.
- Run the environment repair/update step after upgrading so Python Playwright is installed.
- The isolated session is headless; it does not open or reuse the user's normal Edge/Chrome window.

Stop did not end an operating-system process

Stop ends the current model request. Open **Progress > Processes** and terminate the background process separately.

A task was interrupted by restart

Open **Unfinished task** and choose **Continue task**. Pending confirmations, partial visible text, journal state, and persistent process metadata are restored when possible.

Project folder is missing

The project list marks missing directories. Reattach the correct folder, move affected chats, or remove the stale project. Be careful: confirmed project deletion removes the folder and chats rather than merely unregistering it.

Where is an exported PDF or DOCX?

- Project active: `<project>\exports`.
- No project: `sessions\<chat-id>\exports`.
- Research synthesis export: the selected file is also listed in **Results**.

Language did not fully change

Choose **Settings > Appearance and language**. The interface updates in place and preserves the current session. If a native launcher message remains in the old language, restart the desktop application. The saved UI choice takes precedence over the installer-language file.

20. User-Facing Tool Reference

You normally request these operations in natural language; the names below explain what the model can call.

Available in every work mode

Tool	Capability
`read_memory`	Read one complete memory layer.
`save_memory`	Save a durable fact to global, mode, or project memory.
`web_search`	Search current public web results.
`web_fetch`	Read a public HTTP/HTTPS page or supported document.
`context_status`	Inspect tokens, messages, images, pins, and compression.
`pin_context_file`	Pin a text file to the current chat.
`unpin_context_file`	Remove one pin.
`list_project_documents`	List readable documents in the project.
`read_project_document`	Read text, Markdown, PDF, DOCX, JSON, YAML, or CSV.
`list_skills`	List optional skill metadata.
`read_skill`	Load one selected `SKILL.md`.
`export_document`	Export Markdown, DOCX, or PDF.
`list_dir`, `read_file`	Browse directories and read text with line ranges.
`search_files`, `find_files`	Fast literal/regex search and file globs.
`write_file`, `apply_patch`	Create or atomically edit text files.
`make_directory`, `move_file`, `delete_file`	Structured filesystem changes with task rollback.
`list_task_changes`, `undo_task_changes`	Inspect or revert the current task journal.
`view_image`	Attach a local image for visual analysis.

Added in Writing, Development, and Computer

Tool	Capability
`task_plan_status`	Read the persistent operational plan.
`set_task_plan`	Define ordered task steps.
`update_task_step`	Update progress and step results.
`record_task_validation`	Record a validation performed outside automatic checks.

Added in Development and Computer

Tool	Capability
`repo_overview`	Return the automatic repository map.
`project_instructions`	Read hierarchical project guidance for a path.
`project_validation_profile`	List detected/configured checks.
`start_project_check`	Start the primary or selected validation command.
`find_symbol`	Find declarations across supported programming languages.
`document_symbols`	List declarations and line ranges in one source file.
`find_references`	Find fast whole-word symbol uses across the project.
`browser_open`, `browser_snapshot`	Open and semantically inspect an isolated Edge page.
`browser_fill`, `browser_click`, `browser_press`, `browser_wait`	Interact with fresh element refs and wait for UI changes.
`browser_select`, `browser_hover`, `browser_scroll`	Dropdowns, hover states, and scrolling.
`browser_upload`, `browser_download`	Transfer a local file through the page or save a download.
`browser_viewport`	Set a desktop, tablet, or mobile viewport.
`browser_console`, `browser_network`	Inspect browser diagnostics incrementally.
`browser_screenshot`, `browser_close`	Attach the page for Qwen vision or close the session.
`git_status`	Show branch and file status.
`git_diff`	Show working or staged diff.
`git_commit`	Create a local commit from selected/current-task files.
`run_command`	Run a short Bash, PowerShell, or cmd command.
`start_command`	Start a persistent background command.
`poll_command`	Read incremental process output/status.
`send_stdin`	Send input to a background process.
`terminate_command`	Terminate a process and its child tree.

Added in Computer

Tool	Capability
`screenshot`	Capture and attach the primary display.
`get_screen_info`	Read screen and image coordinate dimensions.
`move_mouse`	Move pointer to image coordinates.
`click`	Click or double-click with a selected mouse button.
`scroll`	Scroll at an optional image position.
`type_text`	Type or paste Unicode text.
`press_key`	Press a key or key combination.

21. Practical Request Examples

```
Discussion: Compare these two production approaches without introducing coding procedures.

Research: Research the current state of virtual production stages. Keep contradictory findings,
show uncertainty, and export the final synthesis as PDF.

Writing: Read the project treatment, rewrite act two in the same voice, and save a DOCX.

Development: Inspect this repository, reproduce the reported failure, fix the root cause,
run the relevant tests, and summarize what changed.

Memory: Remember for this project that final renders go to D:\Show\Delivery.

Pinning: Pin docs\ARCHITECTURE.md for this chat.

Computer: Take a screenshot, open the application settings, enable the requested option,
and verify the result with another screenshot.

Export: Save your existing answer as PDF. Do not research again.
```

22. Operational Notes and Limitations

Permanent product boundaries

The following are **explicit non-goals and will never be added** unless the owner personally reverses this product decision:

- Language servers or an LSP runtime/distribution layer. The built-in lightweight

symbol index is the intended code-navigation solution.

- A persistent interactive terminal as a primary workflow. Shell tools remain a backup.
- Parallel model agents, subagents, or multi-model orchestration. One local model owns

the GPU and performs the work sequentially.

- One-million-token context profiles. Context remains within practical local GPU profiles.
- A general plugin host, MCP ecosystem, or broad third-party integration framework.

These items do not belong to Marvin. They are not deferred roadmap work.

- One large model occupies nearly the full GPU, so model inference is intentionally single-slot.
- Read-only tools and background processes can still run concurrently.
- Switching model or KV cache restarts the server and clears its transient prompt cache, but not chat history.
- Context numbers are estimates; exact tokenization and image cost depend on the model/template.
- Model output can be wrong. Verification tools improve confidence but do not guarantee correctness.
- Computer control is limited to the primary monitor and depends on visible UI state.
- Internet extraction may fail on login walls, scripts-only sites, CAPTCHAs, blocked downloads, or unsupported formats.
- Public skills should be reviewed before installation. Skill text guides the model and can influence its actions.
- Git commit is local unless the user explicitly asks for a push and the environment permits it.
- The application is a personal local tool. Maintain backups of irreplaceable project and session data.

23. Extended Workspace Operations

Reading full documents

Ask for a specific PDF page, DOCX block, Excel sheet/cell range, or CSV row range. The document reader returns continuation coordinates so the next part can be read explicitly instead of stopping at the first hundred rows. Excel can expose cached values or formulas; the reader does not recalculate Excel formulas.

Scanned PDF pages and diagrams can be rendered with `view_document_page` and inspected by the current vision-capable model. Existing Word documents can be changed through `edit_word_document`, retaining paragraphs, tables, and run styling.

Recovering original context

Compression is adapted to Discussion, Research, Writing, Development, or Computer. All input chunks are processed, and intermediate summaries are cached. The original messages remain available through `search_chat_history` and `read_chat_history`, including content no longer in the active model context.

Research saves intermediate evidence notes. Interrupted synthesis can reuse those notes. Citation buttons open the original source text, while found but unloaded candidates remain visible without a credibility filter.

Restore points

Open **Results > Restore points** or use `/checkpoint name`. A restore point captures working files while excluding generated dependencies and model/runtime directories. Task snapshots also reconcile file changes made through model commands. **Restore** reports later conflicting edits instead of silently overwriting them, and a partial failure is not marked fully restored.

The compare button beside a restore point lists everything that changed in the workspace since it was taken - modified, deleted and newly created files - and each row opens the diff viewer against the saved version. This needs no model activity, so a restore point doubles as a way to review your own edits. A file created after the restore point has no saved version behind it: it can be viewed but not reverted.

Portable projects

Use **Settings > Data and backups > Export project**. The ZIP contains project files, project memory and skills, conversation history, attachments, and decision references. **Import project** creates a new project directory and new chat IDs, remapping stored attachment paths and source-chat links.

This project archive is separate from the model/runtime backup. The latter still acts as an online-first fallback and can be used explicitly for offline setup.

Source development and releases

End users choose Minimal or Full; Full includes Python 3.12 and runtime dependencies. Node.js is not a runtime prerequisite. Source developers build the frontend with `npm --prefix frontend ci` and `npm --prefix frontend run build`. Python serves the compiled `ui_dist` directory. Windows package versions are fixed in `requirements-windows-py312.lock`.

The workspace is the only interface. The older Gradio surface was removed in 1.16.0, along with the dependency it needed.